

1 Decorated Programs

Doing a complete proof in Hoare Logic can feel overly verbose. The “core” of a proof consists of the preconditions and postconditions surrounding every command; with those, it’s usually possible to infer the complete shape of the proof tree.

Decorating programs is a way to informally write down a Hoare logic proof of correctness. The idea is to insert assertion decorations between every “line” of the program. Using this informal evidence, you can reconstruct the full formal proof.

1.1 Informal Rules

We’ll set out a few rules to check whether a decorated program represents a valid proof using *local consistency* checks. The rules, as you’ll see, are informal reflections of the formal inference rules for Hoare logic.

Skip. For **skip**, the precondition and postcondition should be the same, like this:

$$\begin{array}{c} \{P\} \\ \text{skip} \\ \{P\} \end{array}$$

Sequence. For sequences, a new assertion R appears between the two commands. The two halves $\{P\} \ c_1 \ \{R\}$ and $\{R\} \ c_2 \ \{Q\}$ must be (recursively) locally consistent:

$$\begin{array}{c} \{P\} \\ c_1; \\ \{R\} \\ c_2 \\ \{Q\} \end{array}$$

Assignment. Assignments are locally consistent when the precondition is the same as the postcondition except that it substitutes the assigned expression in for the variable:

$$\begin{array}{c} \{P[a/x]\} \\ x := a \\ \{P\} \end{array}$$

Conditions. An **if** is locally consistent when both branches are locally consistent after adding the branch condition to each:

```
{P}
if b then
  {P ∧ b}
  c1
  {Q}
else
  {P ∧ ¬b}
  c2
  {Q}
{Q}
```

Loops. A **while** command should be decorated with a loop invariant:

```
{P}
while b do
  {P ∧ b}
  c
  {P}
{P ∧ ¬b}
```

Implication. To capture the CONSEQUENCE rule, you can always write a (valid) implication to connect two commands:

$$\{P\} \Rightarrow \{Q\}$$

1.2 Decorating a Program

These informal rules tell you how to *verify* whether a decorated program is correct, but they don't by themselves tell you how to *construct* those decorations (and thereby a proof). You can do this almost mechanically—except for loop invariants, which still require some creativity to concoct.

Here's an example program:

```
while (0 < y) do (
  x := x + 1;
  y := y - 1
)
```

The first step is to decide what we want to prove about this program. So we write a precondition and postcondition for the whole program. Intuitively, the program adds y onto the initial value of the variable x, so we'll assert that:

```

{x = m ∧ y = n ∧ 0 ≤ n}
while (0 < y) do (
  x := x + 1;
  y := y - 1
)
{x = m + n}

```

This program is a **while** command, so the next step is to come up with a loop invariant. We'll set up the structure first. We need an assertion I where we can "connect" the loop's precondition and postcondition to our overall precondition and postcondition using implications, like this:

```

{x = m ∧ y = n ∧ 0 ≤ n} ⇒
{I}
while (0 < y) do (
  {I ∧ 0 < y}
  x := x + 1;
  y := y - 1
  {I}
)
{I ∧ 0 < y} ⇒
{x = m + n}

```

On every iteration of the loop, the variable x is the sum we want, $m + n$, less the current value of y , which is the number of iterations remaining. So we'll define the invariant I like this:

$$I ::= (x = m + n - y) \wedge 0 \leq y$$

The top implication follows because $n - y = 0$, and the bottom implication is valid because $0 < y$ and $0 \leq y$ together imply $y = 0$.

To finish decorating the program, we need an assertion between the two lines in the body of the loop. By the rule for assignments, we know that this must be $I[y - 1/y]$, or:

$$P_1 ::= (x = m + n - (y - 1)) \wedge (0 \leq y - 1)$$

To make the assignment to x locally consistent, then, its precondition must be $P_1[x + 1/x]$, or:

$$P_2 ::= (x + 1 = m + n - (y - 1)) \wedge (0 \leq y - 1)$$

It's straightforward to see that the precondition at the top of the loop, $I \wedge 0 < y$, implies this new assertion P_2 . Now we can write our complete decorated program using these definitions:

$$\begin{aligned}
& \{x = m \wedge y = n \wedge 0 \leq n\} \Rightarrow \\
& \{I\} \\
& \textbf{while } (0 < y) \textbf{ do } (\\
& \quad \{I \wedge 0 < y\} \Rightarrow \\
& \quad \{P_2\} \\
& \quad x := x + 1; \\
& \quad \{P_1\} \\
& \quad y := y - 1 \\
& \quad \{I\} \\
&) \\
& \{I \wedge 0 \not< y\} \Rightarrow \\
& \{x = m + n\}
\end{aligned}$$

2 Weakest Preconditions

Previously, we discussed the issue of completeness—i.e., whether it is possible to derive every valid partial correctness specification using the axioms and rules of Hoare logic. Unfortunately, if treated as a pure deductive system, Hoare logic cannot be complete. To see why, consider the following partial correctness specifications:

$$\{\textbf{true}\} \textbf{skip} \{P\} \qquad \{\textbf{true}\} c \{\textbf{false}\}$$

The first is valid if and only if the assertion P is valid while the second is valid if and only if the command c does not halt.

It turns out that the culprit is the CONSEQUENCE rule,

$$\text{CONSEQUENCE} \frac{\models (P \Rightarrow P') \quad \{P'\} c \{Q'\} \quad \models (Q' \Rightarrow Q)}{\{P\} c \{Q\}}$$

which includes two premises about the validity of implications between the assertions involved. Proving those implications for our two examples would require proving arbitrary logical predicates P and proving that arbitrary IMP programs c terminate. By Gödel’s incompleteness theorem, we know that there is no consistent mathematical proof system that can do the former. Therefore, Hoare logic cannot be complete in that sense.

On the other hand, Hoare logic does enjoy this property:

Theorem. $\forall P, Q \in \text{Assn}, c \in \text{Com}. \models \{P\} c \{Q\} \text{ implies } \vdash \{P\} c \{Q\}.$

This result, due to Cook (1974), is known as the *relative completeness* of Hoare logic. It says that Hoare logic is no more incomplete than the language of assertions—i.e., if we had an oracle that could decide the validity of assertions, then we could decide the validity of partial correctness specifications. In other words, for any valid Hoare triple, there exists a proof in Hoare logic—as long as you have some external way to decide the validity of the premises to the CONSEQUENCE rule.

Cook’s proof of relative completeness depends on the notion of *weakest liberal preconditions*. Given a command c and a postcondition Q , the weakest liberal precondition is the weakest assertion P such that $\{P\} c \{Q\}$ is valid. Here, “weakest” means that any other valid precondition

implies P . That is, P most accurately describes input states for which c either does not terminate or ends up in a state satisfying Q .

Formally, an assertion P is a weakest liberal precondition of c and Q if:

$$\forall \sigma, I. \sigma \models_I P \iff (C \llbracket c \rrbracket \sigma) \text{ is undefined } \vee (C \llbracket c \rrbracket \sigma) \models_I Q$$

We write $wlp(c, Q)$ for the weakest liberal precondition of command c and postcondition Q . From left-to-right, the formula above states that $wlp(c, Q)$ is a valid precondition: $\models \{P\} c \{Q\}$. The right-to-left implication says it is the weakest valid precondition: if another assertion R satisfies $\models \{R\} c \{Q\}$, then R implies P . It can be shown that weakest liberal preconditions are unique modulo equivalence.

We can calculate the weakest liberal precondition of a command as follows:

$$\begin{aligned} wlp(\mathbf{skip}, P) &= P \\ wlp(x := a, P) &= P[a/x] \\ wlp((c_1; c_2), P) &= wlp(c_1, wlp(c_2, P)) \\ wlp(\mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2, P) &= (b \implies wlp(c_1, P)) \wedge (\neg b \implies wlp(c_2, P)) \end{aligned}$$

The definition of $wlp(\mathbf{while } b \mathbf{ do } c, P)$ is slightly more complicated—it encodes the weakest liberal precondition for each iteration of the loop. To give the intuition, first define the weakest liberal precondition for a loop that terminates in i steps as follows:

$$\begin{aligned} F_0(P) &= \mathbf{true} \\ F_{i+1}(P) &= (\neg b \implies P) \wedge (b \implies wlp(c, F_i(P))) \end{aligned}$$

We can then express the weakest liberal precondition using an infinitary conjunction:

$$wlp(\mathbf{while } b \mathbf{ do } c, P) = \bigwedge_i F_i(P)$$

See Winskel Chapter 7 for the details of how to encode the weakest liberal precondition for a while loop as an ordinary assertion.

To check that our definition is correct, we can prove (how?) that it yields a valid partial correctness specification:

Lemma 1.

$$\begin{aligned} \forall c \in \mathbf{Com}, Q \in \mathbf{Assn}. \\ \models \{wlp(c, Q)\} c \{Q\} \text{ and } \forall R \in \mathbf{Assn}. \models \{R\} c \{Q\} \text{ implies } (R \implies wlp(c, Q)) \end{aligned}$$

It is not hard to prove that it also yields a provable specification:

Lemma 2.

$$\forall c \in \mathbf{Com}, Q \in \mathbf{Assn}. \vdash \{wlp(c, Q)\} c \{Q\}$$

Relative completeness follows by a simple argument:

Proof Sketch. Let c be a command and let P and Q be assertions such that the partial correctness specification $\{P\} c \{Q\}$ is valid. By Lemma 1 we have $\models P \implies wlp(c, Q)$. By Lemma 2 we have $\vdash \{wlp(c, Q)\} c \{Q\}$. We conclude $\vdash \{P\} c \{Q\}$ using the CONSEQUENCE rule. $\square$